

Giải thích các sơ đồ thiết kế trong báo cáo

Contents

Giải thích các sơ đồ thiết kế trong báo cáo	1
0. Từ điển stereotype (phần quan trọng nhất khi bị hỏi)	1
1. Sơ đồ kiến trúc phân tầng — KienTrucPhanTang	2
2. Biểu đồ gói phía frontend — goi-fe	3
3. Biểu đồ gói phía backend — goi-be	3
4. Biểu đồ chi tiết gói — Khen thưởng cá nhân hằng năm — so-do-goi-cnhn	4
5. Biểu đồ chi tiết gói — Khen thưởng đơn vị hằng năm — so-do-goi-dvhn	5
6. Biểu đồ lớp module đề xuất khen thưởng — lop-de-xuat (mẫu Strategy)	6
7. Hai biểu đồ tuần tự — seq-de-xuat, seq-phe-duyet	7
8. Các ERD và sơ đồ khác (điểm qua)	7
9. Bảng tra nhanh khi bị hỏi “code thật ở đâu?”	8
10. Vì sao có phần tử gắn stereotype, có phần tử không?	8
11. Các mối quan hệ — ký hiệu, ý nghĩa, và LÀM SAO biết nó là quan hệ đó từ code	9
12. Tóm tắt nhận diện nhanh (in ra mang đi bảo vệ)	11

Giải thích các sơ đồ thiết kế trong báo cáo

Tài liệu tự học để bảo vệ ĐATN. Tập trung vào **sơ đồ gói (package)**, **sơ đồ lớp (class)** và **ý nghĩa stereotype mức thiết kế**. Mọi class/method/đường dẫn trong tài liệu **trích trực tiếp từ mã nguồn dự án** (có ghi file:dòng để tự kiểm chứng), không bịa.

Phạm vi: chỉ các hình kỹ thuật trong báo cáo SOICT_DATN_Tran_Anh_Duc_20220120/ (Chương 4 – Thiết kế kiến trúc). Hình UI/wireframe/use-case/ERD chỉ điểm qua ở cuối.

Render PDF: `pandoc docs/GIAI_THICH_SO_DO_BAO_CAO.md -o giai_thich.pdf --pdf-engine=xelatex -V mainfont="Times New Roman".`

0. Từ điển stereotype (phần quan trọng nhất khi bị hỏi)

Stereotype là nhãn trong cặp ngoặc nhọn « . . . » (UML: guillemet) gắn trên một phần tử để **phân loại vai trò thiết kế** của nó. Nó không thêm thuộc tính/hành vi mới, chỉ nói “phần tử này đóng vai trò gì trong kiến trúc”. Trong báo cáo này stereotype được dùng để **gắn mỗi lớp vào đúng một tầng kiến trúc phân tầng**, nhờ đó nhìn sơ đồ là biết ngay lớp nằm ở tầng nào.

Stereotype	Tầng kiến trúc	Ý nghĩa thiết kế	Tương ứng trong code
«router»	Trình bày	Khai báo endpoint REST, map URL → controller, mắc chuỗi middleware	routes/*.route.ts
«middleware»	Trình bày	Hàm chặn trên đường request, chạy <i>trước</i> controller (xác thực, phân quyền, validate, log)	middlewares/*.ts

Stereotype	Tầng kiến trúc	Ý nghĩa thiết kế	Tương ứng trong code
«controller» (ngầm: lớp trong gói controllers)	Trình bày	Nhận req, gọi service, gói kết quả qua ResponseHelper. Mông, không chứa logic nghiệp vụ	controllers/*.controller.ts
«interface»	Nghịệp vụ	Hợp đồng (contract) — khai báo method bắt buộc, không cài đặt	interface TypeScript
«service» / lớp trong gói services	Nghịệp vụ	Logic nghiệp vụ chính, điều phối: nạp dữ liệu → áp quy tắc → ghi kết quả	services/*.service.ts
«module»	Nghịệp vụ	Hàm thuần gom theo tệp (không phải class instance). Tách khỏi service để dễ kiểm chứng	services/eligibility/*.ts, services/profile/annual.ts
«constants»	Cấu hình	Tập hằng số/bộ cấu hình bất biến (as const)	constants/*.constants.ts
«repository»	Truy cập dữ liệu	Đóng gói toàn bộ truy cập Prisma cho một model; ranh giới duy nhất chạm ORM	repositories/*.repository.ts
«entity»	Dữ liệu	Thực thể dữ liệu = bảng trong DB (Prisma model)	model trong schema.prisma
«enumeration»	Dữ liệu	Tập giá trị rời rạc. Ở dự án này thực chất là cột String + hằng số TS (Prisma không bật native enum)	constants/*.constants.ts

Câu hỏi bấy thường gặp: “Tại sao «enumeration» mà không phải enum thật?” → Prisma trong dự án không dùng native enum. Các cột như role, status, loại_de_xuat, danh_hieu, gioi_tinh, cap_bac là kiểu String trong DB, ràng buộc giá trị bằng **hằng số TypeScript** ở constants/ (vd ROLES, PROPOSAL_STATUS, PROPOSAL_TYPES). Về dạng «enumeration» chỉ để **làm rõ miền giá trị** cho người đọc, đúng về mặt domain.

Phân biệt «service» và «module»: «service» là đối tượng có trạng thái điều phối (gọi repository, gọi service khác, mở transaction). «module» là **tệp hàm thuần** — không truy vấn DB, không side-effect — nên kiểm thử được với mọi bộ đầu vào mà không cần kết nối CSDL. Ví dụ điển hình: checkChainEligibility (quy tắc chuỗi danh hiệu) là «module», còn ProfileService là «service».

1. Sơ đồ kiến trúc phân tầng — KienTrucPhanTang

Hình: Sơ đồ kiến trúc phân tầng (Layered Architecture) bốn tầng phía backend — Mục 4.1.1.

Backend áp dụng **kiến trúc phân tầng 4 tầng**, mỗi tầng chỉ gọi tầng liền kề bên dưới, không vượt tầng:



Đây KHÔNG phải MVC thuần — điểm này hay bị hỏi: - Backend là **API thuần, không có tầng View** (giao diện do frontend Next.js đảm nhiệm, là một client tách biệt). - So với MVC, dự án **bổ sung tầng Service và Repository** (mẫu Controller–Service–Repository) để tách bạch logic nghiệp vụ khỏi truy cập dữ liệu.

Bằng chứng “không vượt tầng” trong code — quy tắc được ép buộc bằng các anti-pattern trong BE-QLKT/CLAUDE.md: - **AP-1**: Controller **cấm** import { prisma } — mọi truy cập DB phải qua service. - **AP-3**: Helper phải **pure**, không được import prisma/service. - Tầng Repository là **ranh giới duy nhất** chạm Prisma.

Trích minh hoạ chuỗi tầng đúng như sơ đồ (controllers/profile.controller.ts:34):

```
// Presentation: Controller chỉ dispatch, không chứa logic
getAnnualProfile = catchAsync(async (req: Request, res: Response) => {
  const params = req.params as PersonnelIdParams;
  const { personnel_id } = params;
  await personnelService.assertCanViewPersonnel(personnel_id, req.user?.role, req.user?.query);
  const result = await profileService.getAnnualProfile(personnel_id); // → Business
  return ResponseHelper.success(res, { message: 'Lấy hồ sơ hằng năm thành công', data: result });
});
```

Tái cấu trúc đáng kể để defend: trước đây Service gọi thẳng prisma.danhHieuHangNam.findMany(...) (logic nghiệp vụ dính chặt vào Prisma). Sau khi tách Repository, Service gọi danhHieuHangNamRepository.findMany(...) → ranh giới rõ, đổi ORM trong tương lai không phải sửa logic nghiệp vụ.

2. Biểu đồ gói phía frontend — **goi-fe**

Hình: *Biểu đồ gói phía frontend* — Mục 4.1.3.

Toàn bộ mã nguồn FE nằm trong gói `src/`, gồm các gói con và 2 gói ngoài (`public`, `node_modules`). Mũi tên nét đứt là **quan hệ phụ thuộc** (`..>`): gói nguồn dùng gói đích.

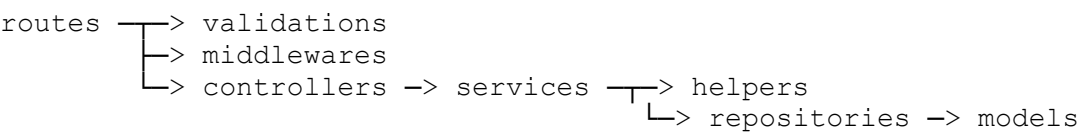
Gói	Ánh xạ thư mục thật	Vai trò
app	FE-QLKT/src/app/	Cây định tuyến Next.js App Router, 4 nhánh theo 4 vai trò (super-admin/admin/manager/user)
components	FE-QLKT/src/components/	Thành phần React dùng chung (auth/, proposals/, personnel/, system-logs/)
api	FE-QLKT/src/lib/api/	Đóng gói mọi request REST tới backend (chia theo miền)
contexts	FE-QLKT/src/contexts/	AuthContext — trạng thái đăng nhập toàn cục
hooks	FE-QLKT/src/hooks/	React hook tự viết: useFetch, useAuthGuard, useSocket
utils	FE-QLKT/src/lib/utils.ts	Hàm tiện ích thuần: formatDate, formatDateTime

Độc luồng phụ thuộc: `app` → `components` → (`api`, `contexts`, `utils`, `hooks`). Tức là **trang gọi component, component mới gọi API/state/utils** — phụ thuộc một chiều, không vòng. `api` được nhiều gói trỏ tới vì nó là **cổng duy nhất** ra backend (quy tắc AP-FE-1: cấm component gọi `fetch/axios` trực tiếp).

3. Biểu đồ gói phía backend — **goi-be**

Hình: *Biểu đồ gói phía backend* — Mục 4.1.3.

Tám gói xếp đúng theo 4 tầng. Chuỗi phụ thuộc:



Gói	Thư mục	Tầng	Vai trò
routes	routes/	Trình bày	Khai báo endpoint, mắc chuỗi middleware
middlewares	middlewares/	Trình bày	auth.ts, auditLog.ts, unitFilter.ts, validate.ts
validations	validations/	Trình bày	Schema Zod cho từng route
controllers	controllers/	Trình bày	Điều hướng req → service → res, mỏng (< 50 dòng/method)
services	services/	Nghiệp vụ	Logic chính (proposal/strategies/, eligibility/, profile/)
helpers	helpers/	Nghiệp vụ	Hàm phụ trợ thuần (auditLog/, notification/, excel/)
repositories	repositories/	Truy cập dữ liệu	Đóng gói Prisma, mỗi model một tệp .repository.ts
models	models/ + schema.prisma	Dữ liệu	Prisma client singleton + 23 model

Chuỗi middleware chuẩn (đọc trên sơ đồ routes → middlewares/validations → controllers) — trích routes/profile.route.ts:

```
import { verifyToken, requireAdminOnly, requireManager } from '../middlewares/auth';

router.get('/annual/:personnel_id', verifyToken, profileController.getAnnualProfile);
router.post('/check-eligibility', verifyToken, requireManager, profileController.checkEligibility);
router.post('/recalculate-all', verifyToken, requireAdminOnly, profileController.recalculateAll);
```

Mẫu tổng quát mỗi route (BE-QLKT/CLAUDE.md): verifyToken → requireAdmin (guard vai trò) → validate(schema) → auditLog(options) → controller.method.

4. Biểu đồ chi tiết gói — Khen thưởng cá nhân hằng năm — so-do-goi-cnhn

Hình: Biểu đồ chi tiết gói nghiệp vụ khen thưởng cá nhân hằng năm — Mục 4.1.4. Đây là module **phức tạp nhất** hệ thống, dùng để minh họa cách 6 tầng phối hợp. **Chú ý các stereotype** vì đây là sơ đồ thể hiện rõ nhất.

Dòng dữ liệu khi xem hồ sơ một quân nhân, đi từ trên xuống:

#	Tầng (stereotype)	Phần tử trên sơ đồ	Code thật
1	«router»	ProfileRoute	routes/profile.route.ts:17 — GET /api/profiles/annual/:personnel_id qua verifyToken

#	Tầng (stereotype)	Phần tử trên sơ đồ	Code thật
2	«middleware»	VerifyToken	middlewares/auth.ts — verifyToken
3	«controller»	ProfileController	controllers/profile.controller.ts — getAnnualProfile
4	«service»	ProfileService	services/profile.service.ts — facade recalculateAnnualProfile (logic thật)
5	«module»	AnnualProfile	services/profile/annual.ts: — recalculateAnnualProfile (logic thật)
6	«module»	PersonalChainEvaluator → ChainEligibility	services/eligibility/chainEligibility.ts
—	«constants»	PersonalChainAwards	constants/chainAwards.constants.ts — PER- SONAL_CHAIN_AWARDS
7	«repository»	QuanNhanRepository, AnnualProfileRepository	repositories/quanNhan.repository.ts repositories/annualProfile.repository.ts
8	«entity»	QuanNhan [thoi đặc] ThanhTichKhoaHoc/DanhHieuHangNam/HoSoHangNam	schema.prisma

Hai điểm thiết kế trên sơ đồ cần nói được khi bảo vệ:

1. **ProfileService («service»)** tách khỏi **ChainEligibility («module»)** — chủ ý. Service chỉ điều phối: nạp DanhHieuHangNam qua repo → gọi computeChainContext → lập PERSONAL_CHAIN_AWARDS gọi checkChainEligibility cho từng cấp → ghi vào bảng suy diễn HoSoHangNam. Còn quy tắc chuỗi là **hàm thuần**, không chạm DB → kiểm thử được mọi kịch bản. Chữ ký thật (services/eligibility/chainEligibility.ts):

```
export function checkChainEligibility(
  award: ChainAwardConfig,
  streaks: ChainStreaks,
  hasReceived: boolean,
  flagsInWindow: FlagsInWindow
): EligibilityResult {
```
2. **Quan hệ composition ([thoi đặc])** từ QuanNhan tới ThanhTichKhoaHoc, DanhHieuHangNam, HoSoHangNam: hồ sơ thành tích/danh hiệu/điều kiện **thuộc vòng đời** một quân nhân (xóa quân nhân → các bản ghi này không tồn tại độc lập). HoSoHangNam là **bảng suy diễn (output cache)** — mỗi quân nhân một bản ghi, tính lại sau mỗi thay đổi nguồn.

5. Biểu đồ chi tiết gói — Khen thưởng đơn vị hằng năm — so-do-goi-dvhn

Hình: Biểu đồ chi tiết gói nghiệp vụ khen thưởng đơn vị hằng năm — Mục 4.1.4 (tiếp).

Cấu trúc tầng **giống** hệ gói cá nhân; khác biệt nằm ở **phân quyền** và **dữ liệu nghiệp vụ**, không ở kiến trúc:

Khác biệt	Gói cá nhân	Gói đơn vị
Middleware phân quyền	chỉ «middleware» VerifyToken (tự xem hồ sơ mình → kiểm ở service)	thêm «middleware» RequireAdminOrManager — chặn vai trò ngay ở route
«constants» cấu hình	PersonalChainAwards	UnitChainAwards
«module» evaluator	PersonalChainEvaluator	UnitChainEvaluator

Khác biệt	Gói cá nhân	Gói đơn vị
«entity»	DanhHieuHangNam, ThanhTichKhoaHoc, HoSoHangNam	DanhHieuDonViHangNam, HoSoDonViHangNam (không có NCKH)
Quy tắc lỗi	dùng chung «module» ChainEligibility	dùng chung «module» ChainEligibility

Điểm vàng để defend (cả 2 sơ đồ cùng chỉ vào một ChainEligibility): quy tắc xét chuỗi danh hiệu được mô tả **đúng một chỗ** (checkChainEligibility). Cá nhân và đơn vị chỉ khác **bộ cấu hình** (PERSONAL_CHAIN_AWARDS vs UNIT_CHAIN_AWARDS). Nhờ vậy hai luồng **không thể lệch nhau**, và thêm danh hiệu mới chỉ cần thêm cấu hình — không sửa logic. Đây là minh họa trực quan của nguyên tắc DRY + Open/Closed.

6. Biểu đồ lớp module đề xuất khen thưởng — 1op-de-xuat (mẫu Strategy)

Hình: Biểu đồ lớp module đề xuất khen thưởng (mẫu Strategy) — Mục 4.1.5. Báo cáo chỉ về **một** sơ đồ lớp (module đề xuất) vì đây là module nghiệp vụ tiêu biểu nhất và minh họa rõ một quyết định thiết kế: **mẫu Strategy (GoF)**.

6.1 Các phần tử trên sơ đồ

Phần tử	Stereotype	Code thật
BangDeXuat	«entity» (ngầm)	schema.prisma model BangDeXuat
LoaiDeXuat	«enumeration»	constants/proposalTypes.constants. — PROPOSAL_TYPES (cột String)
TrangThaiDeXuat	«enumeration»	constants/proposalStatus.constants — PROPOSAL_STATUS
ProposalController	controller	controllers/proposal.controller.ts
ProposalService	service	services/proposal/
ProposalStrategy	«interface»	services/proposal/strategies/propo
ProposalStrategyRegistry	(REGISTRY map)	services/proposal/strategies/index
7 strategy cụ thể	realization .. >	caNhanHangNam/donViHangNam/hccsvv/
SingleMedalImporter	helper dùng chung	services/proposal/strategies/singl
ProposalRepository	«repository»	repositories/proposal.repository.t

6.2 Mẫu Strategy — đọc đúng các đường quan hệ

- **«interface» ProposalStrategy** định nghĩa **1 property + 4 method bắt buộc**. Trích nguyên văn (proposalStrategy.ts:67):

```
export interface ProposalStrategy {
  readonly type: ProposalType;
  buildSubmitPayload(titleData: unknown[], ctx: ProposalSubmitContext): Promise<Submit>
  validateApprove(editedData: EditedProposalData, ctx: ProposalApproveContext): Promise<Submit>
  importInTransaction(editedData, ctx, decisions, pdfPaths, acc, prismaTx): Promise<void>
  buildSuccessMessage(acc: ImportAccumulator): string;
}
```

- **..|> realization** (đường nét đứt + tam giác rỗng): 7 lớp strategy đều **implements** interface. Ví dụ nckhStrategy.ts import đúng các type của interface và khai báo readonly type.
- **--> association ProposalService → ProposalStrategyRegistry**: service **không** dùng if/else theo loại, mà **dispatch qua REGISTRY**. Map thật (strategies/index.ts:17):

```
const REGISTRY: Record<ProposalType, ProposalStrategy | null> = {
  [PROPOSAL_TYPES.CA_NHAN_HANG_NAM]: caNhanHangNamStrategy,
  [PROPOSAL_TYPES.DON_VI_HANG_NAM]: donViHangNamStrategy,
  [PROPOSAL_TYPES.NIEN_HAN]: hccsvvStrategy,
  [PROPOSAL_TYPES.HC_QKQT]: hcqkqtStrategy,
  [PROPOSAL_TYPES.KNC_VSNXD_QDNDVN]: kncStrategy,
  [PROPOSAL_TYPES.CONG_HIEN]: hcbvtqStrategy,
  [PROPOSAL_TYPES.NCKH]: nckhStrategy,
  [PROPOSAL_TYPES.DOT_XUAT]: null,
};
```

Service gọi `getProposalStrategy(type).buildSubmitPayload(...)` thay cho 7 nhánh `if`.

- --> từ **HcqkqtStrategy** và **KncStrategy** tới **SingleMedalImporter**: hai loại “1 quân nhân ↔ 1 huân chương” chia sẻ logic import qua một helper chung để tránh trùng lặp.

6.3 Câu hỏi defend mẫu Strategy

- “Strategy giải quyết vấn đề gì?” → Thay khỏi `switch/if-else` 7 nhánh (dễ phình, khó test) bằng 7 lớp độc lập cùng một hợp đồng. **Open/Closed**: thêm loại đề xuất mới chỉ cần (1) viết `<type>Strategy.ts` implement interface, (2) thêm 1 dòng vào `REGISTRY`, (3) mắc vào `dispatch` — **không sửa** code các loại cũ. `JSDoc` ngay trên interface (`proposalStrategy.ts:62-66`) ghi đúng 3 bước này.
- “Vì sao `DOT_XUAT` map tới `null`?” → Khen thưởng đột xuất do ADMIN ghi **thẳng** vào bảng `KhenThuong-DotXuat` qua `adhocAward.service`, **không đi qua pipeline duyệt BangDeXuat** nên không cần strategy.
- “Submit và Approve khác gì?” → `buildSubmitPayload` (lúc nộp) chỉ validate cấu trúc payload + năm/tháng. Toàn bộ kiểm tra **điều kiện + trùng lặp** chạy ở `validateApprove` (lúc duyệt). Việc import vào bảng đích nằm trong `importInTransaction` (chạy trong transaction Prisma).

7. Hai biểu đồ tuần tự — seq-de-xuat, seq-phe-duyet

Hình: Mục 4.1.6. Hai luồng nghiệp vụ tiêu biểu, khớp với sơ đồ lớp ở §6.

- **Tạo đề xuất** (seq-de-xuat): biểu mẫu 5 bước → `POST /api/proposals` → chuỗi middleware → `proposalService.submitProposal` → `getProposalStrategy(type).buildSubmitPayload` → `proposalRepository.create` → phát `Socket.IO` tới Cán bộ Phòng Chính trị → trả đề xuất.
- **Phê duyệt** (seq-phe-duyet): `POST /api/proposals/:id/approve` → `approveProposal` mở **transaction Prisma** → `validation.preflight` → `strategy.validateApprove` → `decision-Mappings.attach` → `strategy.importInTransaction` → **cập nhật** `BangDeXuat.status = APPROVED` → **tính lại hồ sơ suy diễn** → `commit` → ghi nhật ký → phát `Socket.IO`.

Liên hệ §6: hai bước `strategy.validateApprove` và `strategy.importInTransaction` trên sequence chính là 2 method của «interface» `ProposalStrategy`. Sequence “minh hoạ thời gian”, class “minh hoạ cấu trúc” — cùng một thiết kế.

8. Các ERD và sơ đồ khác (điểm qua)

- **ERD** (`Database/erd-0..6`): 23 bảng, 47 khoá ngoại. Mức 1 = sơ đồ danh mục đủ cột (bỏ đường quan hệ); mức 2 = 6 nhóm nghiệp vụ tách riêng để đường FK hiện rõ. Mỗi «entity» ở các sơ đồ gói/lớp tương ứng một bảng ở đây.
 - Điểm defend: `FileQuyetDinh` là **hub cross-cutting**, 8 bảng khen thưởng trở tới cột `so_quyet_dinh` (cột `UNIQUE`, không phải `id`) — nhờ `ON UPDATE CASCADE`, sửa số quyết định lan truyền tự động.
- **Use case** (uc-*): UC tổng quan + 6 phân rã theo miền + 4 vai trò.
- **Activity** (`activity-quy-trinh`, `activity-excel`): quy trình duyệt đề xuất và luồng nhập Excel.
- **Wireframe / UI** (`wireframe/*`, `UI/*`): bản phác và ảnh chụp giao diện thật theo từng vai trò.

9. Bảng tra nhanh khi bị hỏi “code thật ở đâu?”

Khái niệm trên sơ đồ	Tập nguồn
Kiến trúc phân tầng (ép buộc bằng anti-pattern) «interface» ProposalStrategy	BE-QLKT/CLAUDE.md §Anti-Patterns AP-1, AP-3 BE- QLKT/src/services/proposal/strategies/proposalStrategy.ts
REGISTRY (dispatch Strategy)	BE- QLKT/src/services/proposal/strategies/index.ts
checkChainEligibility («module» thuần)	BE- QLKT/src/services/eligibility/chainEligibility.ts
Cấu hình chuỗi («constants»)	BE- QLKT/src/constants/chainAwards.constants.ts
recalculateAnnualProfile («module» AnnualProfile)	BE-QLKT/src/services/profile/annual.ts
Facade ProfileService («service»)	BE- QLKT/src/services/profile.service.ts
«repository» mẫu	BE- QLKT/src/repositories/quanNhan.repository.ts
Route + chuỗi middleware	BE-QLKT/src/routes/profile.route.ts
23 «entity»	BE-QLKT/prisma/schema.prisma
Sơ đồ lớp chi tiết (bản Mermaid 3 phần)	docs/diagrams/04-class.md
Sơ đồ gói nguồn (PlantUML)	docs/diagrams/03-architecture.md

10. Vì sao có phần tử gắn stereotype, có phần tử không?

Đây là câu hỏi dễ bị vắn. Trả lời gọn: **nhãn «. . .» chỉ xuất hiện khi nhìn cái tên + vị trí gói KHÔNG đủ để biết phần tử đó là gì**. Cụ thể có hai nhóm nhãn, mục đích khác nhau:

10.1 Nhóm 1 — từ khóa UML chuẩn cho phần tử KHÔNG phải class thường

Mặc định mọi ô hộp trong sơ đồ lớp đều được hiểu là **Class**. Khi một ô **không phải** class thường, UML *bắt buộc* phải gắn nhãn để phân biệt **metaclass** (loại phần tử). Trong sơ đồ lớp lop-de-xuat chỉ có 2 loại như vậy:

- «interface» — đây là **hợp đồng**, không có cài đặt. Khác metaclass với Class nên phải ghi rõ.

```
// services/proposal/strategies/proposalStrategy.ts:67
export interface ProposalStrategy {
  readonly type: ProposalType;
  buildSubmitPayload(...): Promise<SubmitValidationResult>;
  // ... 3 method, KHÔNG có thân hàm → đúng nghĩa "interface"
}
```

- «enumeration» — tập giá trị rời rạc. Phải ghi rõ vì nó không phải class.

```
// prisma/schema.prisma:483, 486 (BangDeXuat)
loai_de_xuat String @db.VarChar(20) // CA_NHAN_HANG_NAM, DON_VI_HANG_NAM, NIEN_HAN, C
status String @default("PENDING") @db.VarChar(20) // PENDING, APPROVED, REJECTED

→ Cột là String, miễn giá trị ràng buộc bằng hằng số (PROPOSAL_TYPES, PROPOSAL_STATUS). Vẽ «enu-
meration» để làm rõ domain.
```

Hệ quả: trong sơ đồ lớp, mọi ô còn lại — ProposalController, ProposalService, ProposalRepository, ProposalStrategyRegistry, 7 strategy, SingleMedalImporter, BangDeXuat — đều là **class thường (cùng metaclass)** nên **không cần nhãn**; vai trò của chúng đọc qua **tên + quan hệ**, không cần stereotype.

10.2 Nhóm 2 — stereotype tùy biến cho vai trò kiến trúc (trong sơ đồ gói chi tiết)

Sơ đồ gói chi tiết (so-do-goi-cnhn, so-do-goi-dvhn) trộn nhiều **loại tạo phẩm không thuần là class OOP**: một tệp route, một tệp hàm thuần, một tệp hằng số, một thực thể DB. Khi đó nhân được dùng để **chỉ rõ tầng/vai trò** vì cái tên không tự nói lên:

Ô có nhãn	Vì sao cần nhãn (tên không tự rõ)
«router» ProfileRoute	“ProfileRoute” có thể là class bất kỳ → «router» nói rõ là tệp khai báo endpoint
«middleware» VerifyToken	“VerifyToken” nghe như một hàm/biến → «middleware» nói rõ nó chặn trên đường request
«module» ChainEligibility, «module» AnnualProfile	Đây là tệp hàm thuần , không phải class có instance → «module» phân biệt với service
«constants» PersonalChainAwards	Là bộ cấu hình bất biến, không phải logic
«repository» QuanNhanRepository	Nhấn mạnh đây là ranh giới chạm Prisma
«entity» QuanNhan	Là bảng DB, không phải class nghiệp vụ thường

10.3 Vậy vì sao ProfileController / ProfileService trong sơ đồ gói chi tiết LẠI KHÔNG có nhãn?

Vì hai điều kiện đã làm vai trò **hiển nhiên**, gán thêm nhãn là thừa: 1. Chúng nằm **trong gói tên controllers / services** (vị trí đã nói lên tầng), và 2. Tên có **hậu tố Controller / Service**.

```
// controllers/profile.controller.ts:33 - tên + gói đã đủ, không cần «controller»
class ProfileController {
  getAnnualProfile = catchAsync(async (req, res) => { ... });
}
// services/profile.service.ts:233
export default new ProfileService();
```

Quy tắc nhất quán mà các sơ đồ tuân theo: Chỉ gán nhãn khi (a) phần tử khác metaclass với class thường (interface/enum), HOẶC (b) tên + gói chưa đủ để suy ra vai trò. Class nghiệp vụ thường có tên rõ ràng thì để trống.

11. Các mối quan hệ — ký hiệu, ý nghĩa, và LÀM SAO biết nó là quan hệ đó từ code

Trong các sơ đồ có **4 loại quan hệ**. Cách nhận biết: trước hết nhìn **ký hiệu đường vẽ**, rồi đối chiếu **bằng chứng trong code** (chính code quyết định loại quan hệ, không phải vẽ tùy ý).

11.1 Realization / Hiện thực hóa — ..|> (nét đứt + tam giác RỖNG)

- **Ý nghĩa:** một class *cài đặt* một interface (cung cấp đúng hợp đồng).
- **Ở đâu:** 7 strategy -> ProposalStrategy.
- **Làm sao biết:** code dùng đúng từ khóa implements. Từ khóa implements của TypeScript **chính là** realization trong UML — không thể nhầm.

```
// services/proposal/strategies/nckhStrategy.ts:61
class NckhStrategy implements ProposalStrategy {
  readonly type = PROPOSAL_TYPES.NCKH;
  // ... cài đặt đủ 4 method của interface
}
export const nckhStrategy = new NckhStrategy(); // dòng 224

// services/proposal/strategies/hccsvvStrategy.ts:26
class HccsvvStrategy implements ProposalStrategy {
  readonly type = PROPOSAL_TYPES.NIEN_HAN;
}
```

- **Tại sao realization mà KHÔNG phải generalization (kế thừa)?** Các strategy chỉ chia sẻ **một hợp đồng**, không chia sẻ cài đặt; chúng implement một interface, không kế thừa class cha. Generalization (đường --|> nét liền + tam giác rỗng tới superclass) **không xuất hiện** trong sơ đồ này. Đây là điểm hay bị hỏi: “sao không cho strategy kế thừa một BaseStrategy?” → vì không có hành vi chung đủ lớn để gom; phần dùng chung (HCQKQT + KNC) được tách thành **helper** singleMedalImporter, không phải lớp cha.

11.2 Association / Kết hợp — đường nét LIỀN, mũi tên mở 1 chiều

- **Ý nghĩa:** class nguồn **giữ tham chiếu ổn định** tới class đích và dùng nó lâu dài (một field, hoặc một singleton được import).
- **Ở đâu:** ProposalController → ProposalService, ProposalService → ProposalRepository, ProposalService → ProposalStrategyRegistry, ProposalRepository → BangDeXuat.
- **Làm sao biết:** module nguồn import đối tượng đích (singleton ổn định) rồi gọi method **nhiều lần** qua tham chiếu đó.

```
// services/proposal/approve.ts:1 - import tham chiếu ổn định
import { proposalRepository } from '../..repositories/proposal.repository';
// ...gọi lặp lại qua tham chiếu:
proposalRepository.findUniqueRaw({ ... }); // dòng 59
proposalRepository.updateMany( ... ); // dòng 375

// controllers/profile.controller.ts - controller giữ và gọi service
import profileService from '../services/profile.service';
const result = await profileService.getAnnualProfile(personnel_id);
```
- **Tại sao mũi tên 1 chiều (navigability)?** Vì kiến trúc phân tầng: phụ thuộc **chỉ đi xuống** (Controller→Service→Repository→Entity), tầng dưới **không biết** tầng trên. Controller import service, nhưng service **không** import controller → mũi tên một chiều, hướng xuống. Đây chính là lý do “vì sao mối quan hệ lại như vậy”.

11.3 Dependency / Phụ thuộc — . .> (nét ĐÚT, mũi tên mở)

- **Ý nghĩa:** dùng *thoảng qua* — đối tượng đích chỉ là **giá trị trả về** / **tham số** / **biến cục bộ**, nguồn **không giữ** tham chiếu lâu dài.
- **Ở đâu:** ProposalStrategyRegistry . .> ProposalStrategy (registry *trả về* một strategy); Hc- qkqtStrategy / KncStrategy dùng SingleMedalImporter (gọi một hàm).
- **Làm sao biết:** đối tượng đích xuất hiện ở **kiểu trả về** hoặc **lời gọi hàm**, không phải field được lưu.

```
// services/proposal/strategies/index.ts:33 - TRẢ VỀ strategy (dùng thoảng qua)
export function getProposalStrategy(type: ProposalType): ProposalStrategy | null {
  return REGISTRY[type] ?? null;
}

// services/proposal/strategies/hcqkqtStrategy.ts:17, 88 - GỌI hàm helper trong transa
import { importSingleMedal } from './singleMedalImporter';
// ...bên trong importInTransaction():
await importSingleMedal(nienHanData, ctx, acc, prismaTx, { ... });
```
- **Phân biệt với association (rất hay hỏi):** giữ tham chiếu lâu dài (field/singleton import dùng nhiều nơi) = association; chỉ chạm tới qua param/return/biến cục bộ rồi bỏ = dependency. singleMedalImporter được gọi như một hàm trong thân method nên mang tính dependency; một số sơ đồ vẽ bằng đường có hướng để nhấn “có dùng”.

11.4 Composition / Cấu thành — đường nét LIỀN + hình thoi ĐẶC ([thoi đặc]) ở đầu “toàn thể”

- **Ý nghĩa:** quan hệ toàn thể–bộ phận **có sở hữu vòng đời**: bộ phận không tồn tại độc lập, xóa toàn thể thì bộ phận mất theo.
- **Ở đâu:** trong so-do-goi-cnbn: QuanNhan [thoi đặc]— ThanhTichKhoaHoc / DanhHieuHangNam / HoSoHangNam.

- **Làm sao biết:** trong schema, các bảng con có FK trở về **QuanNhan** và **không có ý nghĩa nếu thiếu quân nhân** (một bản ghi HoSoHangNam hay DanhHieuHangNam không có danh tính độc lập — nó luôn thuộc về đúng một quân nhân, HoSoHangNam còn là mỗi quân nhân đúng một bản ghi suy diễn).

```
// prisma/schema.prisma — bản ghi con thuộc vòng đời QuanNhan
model DanhHieuHangNam {
  quan_nhan_id String
  QuanNhan      QuanNhan @relation(fields: [quan_nhan_id], references: [id])
  // không có quan_nhan_id thì bản ghi vô nghĩa → composition
}
```

- **Tại sao là composition chứ không phải association?** So sánh với QuanNhan → ChucVu: một ChucVu (chức vụ) **tồn tại độc lập**, nhiều quân nhân dùng chung, xóa quân nhân không xóa chức vụ → đó là **association**. Còn hồ sơ/danh hiệu của quân nhân thì gắn chặt vòng đời → **composition**. Chính **ngữ nghĩa sở hữu vòng đời** quyết định, không phải hình vẽ.

11.5 Dependency giữa GÓI (trong sơ đồ gói) — `..>` nét đứt

Trong `goi-fe / goi-be`, mũi tên nét đứt giữa các **gói** cũng là **dependency**: gói nguồn dùng (import) gói đích. Ví dụ `controllers ..> services ..> repositories ..> models`. Cách biết: mã trong gói nguồn import từ gói đích. Quy tắc một chiều, xuống tầng — giống §11.2.

12. Tóm tắt nhận diện nhanh (in ra mang đi bảo vệ)

Quan hệ	Ký hiệu	Bảng chứng code	Ví dụ trong dự án
Realization	nét đứt + tam giác rỗng <code>.. ></code>	từ khóa <code>implements</code>	<code>class NckhStrategy implements ProposalStrategy</code>
Association	nét liền + mũi tên mở	<code>field / singleton import</code> dùng lâu dài	<code>ProposalService giữ proposalRepository</code>
Dependency	nét đứt + mũi tên mở <code>..></code>	<code>param / return / biến cục bộ</code>	<code>getProposalStrategy() : ProposalStrategy</code>
Composition	nét liền + thoi đặc [thoi đặc]	FK + bộ phận không có danh tính độc lập	<code>QuanNhan [thoi đặc] DanhHieuHangNam</code>
Generalization (KHÔNG dùng)	nét liền + tam giác rỗng tới superclass	từ khóa <code>extends</code>	— (dự án không kế thừa class)

Ba câu chốt khi bị hỏi: 1. *Stereotype để làm gì?* → phân loại vai trò/tầng kiến trúc; chỉ gắn khi tên + gói chưa tự nói rõ, hoặc khi khác metaclass (interface/enum). 2. *Sao biết là realization?* → vì code dùng `implements interface`, không phải `extends class`. 3. *Sao các mũi tên đều một chiều xuống?* → kiến trúc phân tầng, phụ thuộc chỉ đi xuống, tầng dưới không biết tầng trên.